P vs NP Grassroots: Ground-Up Background Theory for the Goertzel Program

Oruzi / Codex collaboration

March 28, 2026

Abstract

This note records the *grassroots* side of the current Lean work around Ben Goertzel's proposed route to $P \neq NP$. It is intentionally not the blocker note. The companion crux-first analysis lives separately in `pnp_crux.tex`.

The goal here is narrower and more constructive: build reusable background theory from the ground up, using mathlib and existing Mettapedia infrastructure wherever possible, so that if the high-level proof strategy can be repaired, it lands in a clean machine-checked framework rather than in ad hoc prose.

The current grassroots state now includes explicit encoded hypothesis classes, finite empirical-risk minimization, finite deceptive-sample bounds, threshold recovery lemmas, a rational uniform-rate layer, and explicit uniform success bounds for exact ERM recovery under finite sampling. It now also includes the first weighted finite-sampling steps beyond uniform draws: an exact one-predictor agreement law and a family-level weighted union bound over finite encoded classes, together with weighted exact-recovery lower bounds for ERM. It also now packages the exact *same-route interface*: a repair that still intends to use switching plus ERM must produce a bit-encoded family over an available input surface and therefore inherits the current finite weighted recovery theorem. The Meredith-side limit is now considerably sharper. The currently formalized weakness bridge exports only global evidence scalars, so any predictor factoring only through those summaries is constant on the whole state space. The current exact HML layer is infinitary by default: if the certified minimal-context type is infinite, then even the depth-1 fragment is infinite, and the concrete rho instance already realizes that case. And one layer below exact HML, the concrete rho present-moment surface already realizes arbitrary finite membership patterns on a natural probe slice: a single-output probe state against a flat bag of input guards records exactly which channels are present in the environment. On the semantic side, it now includes a depth-bounded GSLT/HML observational kernel, a bridge from bisimulation quotients to quantale weakness, and a first concrete minimal-context instance for the rho calculus. This isolates the exact positive shape that a successful switching theorem would need: not mere locality, but locality together with a concrete compression theorem, a clean finite learning interface, and an explicit observational/evidence semantics.

Contents

1 Scope **2**

2 Existing Lean Base **3**

3 Encoded Hypothesis Families **3**

4 Finite-Class ERM Kernel **4**

5 Finite Uniform Consistency Bound **4**

6 Finite Uniform Recovery Threshold **5**

7 Finite Uniform Rate Layer **5**

8 Finite Uniform Success Bounds **6**

9 Finite Weighted Agreement and Family Bounds **7**

10 Finite Weighted Recovery Bounds **8**

11 What Still Counts as the Same Route **8**

12 Hyperseed as a Local-View Surface **9**

13 Operational Distinction and Weakness **10**

14 Kolmogorov Foundations Already Present **12**

15 What the Grassroots Program Needs Next **12**

15.1 Finite-Class Learning Layer 13

15.2 Concrete Decoder Subclasses 13

15.3 Bridging to the Full PNP Route 14

16 Current Status **14**

17 Conclusion **14**

1 Scope

The October 2025 Goertzel manuscript *"P != NP: A Non-Relativizing Proof via Quantale Weakness and Geometric Complexity"* proposes a route from a structured Unique-SAT ensemble to a distributional lower bound, and then to a contradiction with the upper bound that would follow from $P = NP$.

This document does *not* ask where that route fails. That is the job of the companion crux note. Instead it asks a different question:

> What background theory would we want in Lean even if the final proof still needs repair?

That question leads to a program with three design constraints:

1. Prefer general infrastructure over one-off rebuttal lemmas.
2. Reuse mathlib computability and existing Mettapedia semantics rather than inventing parallel foundations.
3. State optimistic bridge theorems in a way that makes missing hypotheses explicit, so later fixes can slot into them cleanly.

2 Existing Lean Base

Several ingredients already exist in the local formalization and are useful for a ground-up PNP effort.

Module	Grassroots role
`Mathlib.Computability.Partrec`	partial recursive computation infrastructure
`KolmogorovComplexity/Basic.lean`	universal algorithm and plain complexity
`Hyperseed/Ultrainfinitism.lean`	observer-relative available-region semantics
`Hyperseed/Basic.lean`	closure/cascade interfaces on accessible observations
`GSLT/Logic/LogicalMetric.lean`	finite-depth observational distinction kernel
`GSLT/Meredith/WeaknessBridge.lean`	operational distinguishability → weakness/evidence
`GSLT/Meredith/RhoMinimalContext.lean`	first concrete minimal-context / HML anchor
`PNP/HypothesisClass.lean`	encoded classifier families and code-budget bounds
`PNP/FiniteERM.lean`	empirical-risk minimization kernel for finite encoded families
`PNP/FiniteConsistencyBound.lean`	finite uniform consistency bound for deceptive samples
`PNP/FiniteUniformRecovery.lean`	threshold corollary guaranteeing exact recovery on some sample
`PNP/FiniteUniformRate.lean`	rational uniform-rate bounds for deception and exact recovery
`PNP/FiniteUniformSuccess.lean`	miss-ratio decay and explicit uniform success bounds
`PNP/FiniteIIDAgreement.lean`	exact weighted one-predictor agreement mass under a finite PMF
`PNP/FiniteIIDFamilyBound.lean`	weighted deceptive-mass union bound for finite encoded families
`PNP/FiniteIIDRecovery.lean`	weighted exact ERM recovery lower bounds under finite PMFs
`PNP/SameRouteInterface.lean`	exact certificate for when a repair still counts as the same switchi
`PNP/GlobalWeaknessObstruction.lean`	current weakness summaries alone cannot yield a nonconstant pre
`PNP/InfinitaryHMLObstruction.lean`	infinite minimal contexts force an infinite depth-1 HML observabl
`PNP/PresentMomentShattering.lean`	exact rho present-moment probes already realize arbitrary finite m

The point of the table is simple: the grassroots route is not starting from zero. We already have a serious computability substrate, an observer-relative semantics layer, and a new optimistic interface for small hypothesis classes.

3 Encoded Hypothesis Families

The key new grassroots abstraction is that a hypothesis class should be represented by a finite code space together with a decoder.

code budget first, realized class second.

Formally, `HypothesisClass.lean` introduces an *encoded family*

$$\text{decode} : \text{Code} \to (\text{Input} \to \text{Output}),$$

where Code is finite. The realized hypothesis class is the range of decode.

Theorem 3.1 (Realized class is bounded by code space)**.** *For any encoded family H,*

$$|\text{realized}(H)| \leq |\text{Code}(H)|.$$

This is the theorem `EncodedFamily.card_realized_le` in Lean.

The specialization relevant to the Goertzel program is the bit-coded case.

Theorem 3.2 (Bit-budget bound)**.** *If a family of Boolean classifiers is decoded from s bits, then it realizes at most 2^s distinct classifiers.*

This is formalized as `BitEncodedClassifierFamily.card_realized_le`.

This theorem is elementary, but it captures the exact shape of the optimistic repair that would be needed later. A switching theorem cannot stop at "the new predictor is local." It must continue to a statement of the form:

> the switched predictors lie in a family decoded from $\mathrm{polylog}(m)$ bits.

Only then does finite-class learning theory become genuinely available.

4 Finite-Class ERM Kernel

The next honest step after a finite encoded family is not yet a PAC theorem. It is the finite combinatorial kernel of empirical risk minimization.

The new module `FiniteERM.lean` defines labeled samples as finite lists of pairs (x, y), empirical error as the number of misclassified examples in the sample, and exact sample consistency as zero empirical error.

Theorem 4.1 (Finite ERM existence)**.** *Let H be a nonempty encoded family. For every finite labeled sample S, there exists a realized predictor $h^\star \in \mathrm{realized}(H)$ such that*

$$\mathrm{err}_S(h^\star) \le \mathrm{err}_S(g) \qquad \textit{for all } g \in \mathrm{realized}(H).$$

This is formalized as `EncodedFamily.exists_realized_empiricalRiskMinimizer`.

Theorem 4.2 (Zero-error transfer through ERM)**.** *If some code in the family fits the sample exactly, then the chosen ERM predictor also fits the sample exactly.*

This is formalized as `EncodedFamily.empiricalRiskPredictor_fitsSample_of_exists_code_fits`.

This is still modest, but it matters. Combined with the code-budget theorem from `HypothesisClass.lean`, it gives the first clean bridge from "small encoded family" to an actual learning-theoretic construction.

5 Finite Uniform Consistency Bound

The next step now formalized is a fully combinatorial finite-domain bound. We do not yet assume an arbitrary distribution or invoke measure theory. Instead, we count how many length-m input samples can make a wrong predictor look perfectly consistent with the target labels.

Theorem 5.1 (Wrong predictors fit only exponentially few samples)**.** *Let X be a finite input domain. If $h \neq f$, then the number of length-m samples on which h agrees with the target f at every sampled point is at most*

$$(|X| - 1)^m.$$

This is formalized as `card_consistentSamples_le_of_exists_disagreement` in `FiniteConsistencyBound.lean`.

Theorem 5.2 (Finite encoded families have few deceptive samples)**.** *Let H be an encoded family over a finite input domain X. The number of length-m samples on which some wrong code in H still matches all target labels is at most*

$$|\mathrm{Code}(H)| \cdot (|X| - 1)^m.$$

This is formalized as `EncodedFamily.card_deceptiveSamples_le`.

Theorem 5.3 (ERM recovers the target off the deceptive set)**.** *If the target predictor is realized by the encoded family, then on every non-deceptive sample the chosen ERM predictor equals the target predictor exactly.*

This is formalized as `EncodedFamily.empiricalRiskPredictor_eq_target_of_not_deceptive`.

This is not yet the full finite-class generalization theorem one would state in terms of $\log |H|$ and an arbitrary data distribution. But it is already the right union-bound skeleton in a completely concrete finite setting.

6 Finite Uniform Recovery Threshold

The next corollary is the cleanest possible extraction from the deceptive-sample bound. If the deceptive set is strictly smaller than the whole length-m sample space, then at least one non-deceptive sample exists.

Theorem 6.1 (Threshold for existence of a good sample)**.** *Let H be an encoded family over a finite input domain X. If*

$$|\mathrm{Code}(H)| \cdot (|X| - 1)^m < |X|^m,$$

then there exists a length-m sample outside the deceptive set.

This is formalized as `EncodedFamily.exists_nondeceptiveSample_of_bound_lt` in `FiniteUniformRecovery.lea`

Theorem 6.2 (Threshold corollary for exact ERM recovery)**.** *If the target predictor is realized by the encoded family and*

$$|\mathrm{Code}(H)| \cdot (|X| - 1)^m < |X|^m,$$

then there exists a length-m sample on which the chosen ERM predictor equals the target predictor exactly.

This is formalized as `EncodedFamily.exists_sample_empiricalRiskPredictor_eq_target_of_bound_lt`.

This is still finite and non-asymptotic, but it clarifies the next job. To move from existence to a real learning theorem, we now need a probability interface that turns the combinatorial ratio behind the threshold into an explicit success bound under sampling assumptions.

7 Finite Uniform Rate Layer

The next natural refinement is still finite, but no longer purely existential. Instead of asking only whether some good sample exists, we measure how large the good and bad regions are inside the finite sample space.

The module `FiniteUniformRate.lean` defines rational uniform densities on the point-sample space:

1. the deceptive rate,

2. the non-deceptive rate,

3. the exact-recovery rate for ERM.

Theorem 7.1 (Uniform deceptive-rate bound)**.** *Let H be an encoded family over a nonempty finite input domain X. Then the uniform fraction of length-m samples that are deceptive is at most*

$$\frac{|\mathrm{Code}(H)| \cdot (|X| - 1)^m}{|X|^m}.$$

This is formalized as `EncodedFamily.deceptiveRate_le_bound` in `FiniteUniformRate.lean`.

Theorem 7.2 (Exact recovery dominates the non-deceptive region)**.** *If the target predictor is realized by the encoded family, then the uniform exact recovery rate of the chosen ERM predictor is at least*

$$1 - \mathrm{rate}_{\mathrm{deceptive}}.$$

This is formalized as `EncodedFamily.exactRecoveryRate_ge_one_sub_deceptiveRate`.

Theorem 7.3 (Positive exact recovery rate under the threshold)**.** *If the target predictor is realized and*

$$|\mathrm{Code}(H)| \cdot (|X| - 1)^m < |X|^m,$$

then the exact recovery rate is strictly positive.

This is formalized as `EncodedFamily.exactRecoveryRate_pos_of_bound_lt`.

This is still not a full PAC-style theorem under arbitrary distributions. But it does close the uniform finite-sampling gap between "some good sample exists" and "good samples occupy an explicitly bounded fraction of the sample space."

8 Finite Uniform Success Bounds

The next cleanup step is to express the finite uniform rate bound using the single-step miss ratio

$$\rho_X := \frac{|X| - 1}{|X|}.$$

For any nonempty finite input domain, $\rho_X < 1$, so the finite deceptive rate decays exponentially in the sample length m.

The module `FiniteUniformSuccess.lean` packages the previous rate theorem into this cleaner interface.

Theorem 8.1 (Miss-ratio deceptive bound)**.** *Let H be an encoded family over a nonempty finite input domain X. Then*

$$\mathrm{rate}_{\mathrm{deceptive}} \le |\mathrm{Code}(H)| \cdot \rho_X^m.$$

This is formalized as `EncodedFamily.deceptiveRate_le_codeMul_uniformMissRatio_pow`.

Theorem 8.2 (Uniform exact recovery lower bound)**.** *If the target predictor is realized by the encoded family, then*

$$\mathrm{rate}_{\mathrm{exact\ recovery}} \ge 1 - |\mathrm{Code}(H)| \cdot \rho_X^m.$$

This is formalized as `EncodedFamily.exactRecoveryRate_ge_one_sub_codeMul_uniformMissRatio_pow`.

Theorem 8.3 (ε-style corollary)**.** *If*

$$|\mathrm{Code}(H)| \cdot \rho_X^m \le \varepsilon,$$

then the exact recovery rate is at least $1 - \varepsilon$*.*

This is formalized as `EncodedFamily.exactRecoveryRate_ge_one_sub_of_codeMul_uniformMissRatio_pow_le`.

This is still uniform finite-sample learning rather than full distributional generalization. But it is the first version of the grassroots story with an explicit success lower bound of the form "one minus an exponentially decaying failure term."

9 Finite Weighted Agreement and Family Bounds

The next honest move is to relax the uniform-sampling assumption while still staying fully finite. Instead of sampling uniformly from a finite input domain, we let one finite probability mass function μ weight the inputs.

The module `FiniteIIDAgreement.lean` handles the one-predictor case. For one target f and one predictor h, it defines:

1. the one-step agreement mass of h with f under μ,
2. the product mass of a length-m sample under independent draws from μ,
3. the total mass of samples on which h agrees with f everywhere.

Theorem 9.1 (Exact weighted agreement law)**.** *For any finite input type and finite PMF* μ*,*

$$\mathrm{mass}_\mu(\textit{all-}m\textit{-point samples consistent with } h) = \left(\mathrm{agreementMass}_\mu(f,h)\right)^m.$$

This is formalized as `consistentSampleMass_eq_agreementMass_pow`.

Theorem 9.2 (Weighted decay corollary)**.** *If the one-step agreement mass is at most* q*, then the total mass of length-*m *consistent samples is at most* q^m*.*

This is formalized as `consistentSampleMass_le_pow_of_agreementMass_le`.

The new module `FiniteIIDFamilyBound.lean` lifts this to the encoded-family level.

Theorem 9.3 (Weighted deceptive-family union bound)**.** *Let* H *be a finite encoded family and* μ *a finite PMF on the input type. Then the total* μ^m*-mass of deceptive samples is at most*

$$\sum_{c \in \mathrm{BadCodes}(H,f)} \left(\mathrm{agreementMass}_\mu(f, \mathrm{decode}(c))\right)^m.$$

This is formalized as `EncodedFamily.deceptiveSampleMass_le_badCodeAgreementMassSum`.

Theorem 9.4 (Uniform weighted family corollary)**.** *If every bad code in the family has one-step agreement mass at most* q*, then the total deceptive mass is at most*

$$|\mathrm{Code}(H)| \cdot q^m.$$

This is formalized as `EncodedFamily.deceptiveSampleMass_le_codeCard_mul_pow_of_agreementMass_le`.

This is the first genuinely family-level finite distributional theorem in the grassroots development. The next real probabilistic step is no longer a union bound itself, but a recovery or generalization theorem built on top of this weighted family bound.

10 Finite Weighted Recovery Bounds

The module `FiniteIIDRecovery.lean` closes the next natural loop. It turns the weighted deceptive-mass bound into a weighted success statement for the chosen ERM predictor.

Theorem 10.1 (Weighted exact recovery dominates the nondeceptive mass)**.** *If the target predictor is realized by the encoded family, then the total μ^m-mass of exact ERM recovery is at least*

$$1 - \mathrm{deceptiveMass}_\mu.$$

This is formalized as `EncodedFamily.exactRecoverySampleMass_ge_one_sub_deceptiveSampleMass`.

Theorem 10.2 (Weighted exact recovery from bad-code agreement masses)**.** *If the target predictor is realized by the encoded family, then the exact recovery mass is at least*

$$1 - \sum_{c \in \mathrm{BadCodes}(H,f)} \left(\mathrm{agreementMass}_\mu(f, \mathrm{decode}(c))\right)^m.$$

This is formalized as `EncodedFamily.exactRecoverySampleMass_ge_one_sub_badCodeAgreementMassSum`.

Theorem 10.3 (Uniform weighted exact recovery corollary)**.** *If every bad code has one-step agreement mass at most q, then the exact recovery mass is at least*

$$1 - |\mathrm{Code}(H)| \cdot q^m.$$

This is formalized as `EncodedFamily.exactRecoverySampleMass_ge_one_sub_codeCard_mul_pow_of_agreement`

At this point the grassroots development has reached a fully finite, distribution-weighted recovery theorem for ERM over encoded families. The next step is to lift this from exact recovery on a finite sample space to a genuine finite-class generalization statement.

11 What Still Counts as the Same Route

At this stage the grassroots theory can also say something more conceptual, but in a machine-checked way. If a proposed rescue still wants to be *the same route* as the current switching/ERM program, then it cannot merely gesture at weakness, GSLTs, or observer-relative semantics. It must still instantiate the existing finite learning interface.

The new module `SameRouteInterface.lean` packages exactly that point. A *same-route family* is just:

1. one available input surface $\mathrm{AvailableInput}(P, B, G)$,
2. one s-bit encoded boolean predictor family on that surface,
3. one target predictor realized by some code in that family.

Theorem 11.1 (Same-route weighted recovery certificate)**.** *Let F be an s-bit encoded classifier family and suppose the target predictor is realized by some code in F. If every bad code has one-step agreement mass at most q, then the exact ERM recovery mass is at least*

$$1 - 2^s q^m.$$

This is formalized as `BitEncodedClassifierFamily.exactRecoverySampleMass_ge_one_sub_bitBudget_mul_pc`

The point is not that this theorem proves the Goertzel switching step. The point is that it now marks the boundary of the current route. A weakness/GSLT idea that still intends to be a *repair* of the present program must hand over this sort of certificate: an explicit visible input surface, an explicit small code space, and explicit agreement-mass control for the bad codes. If it does not, then it is no longer filling a gap in the current route. It is starting a different route.

The new module `GlobalWeaknessObstruction.lean` makes the first exact Meredith-side pressure point equally explicit. In the current Lean bridge, `distinctionWeakness ev` and `nonDistinctionWeakness ev` are global scalars attached to the whole finite evidence assignment ev. They do not vary with the underlying state $u \in U$.

Theorem 11.2 (Weakness-only factoring is constant)**.** *If a boolean predictor $h : U \to \{\text{false}, \text{true}\}$ factors through either of the current global summaries*

$$\text{distinctionWeakness}(ev) \qquad \textit{or} \qquad \text{nonDistinctionWeakness}(ev),$$

then h is constant on U:

$$\forall u, v \in U,\ h(u) = h(v).$$

This is formalized as `factorsThroughDistinctionWeakness_constant` and `factorsThroughNonDistinctionWeak`
So the current weakness bridge is real semantic infrastructure, but by itself it does not yet instantiate the same-route learning interface. Any GSLT/weakness rescue that still wants to count as the current switching/ERM route must add genuinely per-instance observables or side information beyond these present global summaries.

12 Hyperseed as a Local-View Surface

The existing Hyperseed formalization already provides a clean observer-relative semantics.

Given a perspective P, a budget B, and a guard region G, it defines an *available region*

$$\text{availableRegion}(P, B, G) = \text{observableUniverse}(P) \cap \text{nearEurycosm}(P, B) \cap G.$$

For the grassroots PNP program, this matters because it gives a principled way to talk about what a decoder can even access. Instead of treating locality as an informal phrase, we can work with a typed input surface:

$$\text{AvailableInput}(P, B, G) := \{x : \text{World} \mid x \in \text{availableRegion}(P, B, G)\}.$$

The new Lean module proves the corresponding encoded-family bound on such an available input surface:

Theorem 12.1 (Available-region code-budget bound)**.** *An s-bit classifier family over one Hyperseed available region still realizes at most 2^s classifiers.*

This is formalized as `card_realized_availableFamily_le_of_bitBudget`.

The theorem is intentionally modest. It does not claim that the Goertzel local view *is* a Hyperseed perspective. It says that Hyperseed already gives us a good general language for bounded observational access, and that this language can interoperate with the encoded-hypothesis-class infrastructure.

13 Operational Distinction and Weakness

My own mathematical and computationalist intuition is that if there is an elegant route through the real blockers, it will likely come from a semantics of *distinguishability* rather than from ad hoc local-rule counting alone. That is the reason the Meredith lane is worth treating as part of the grassroots background theory, at least optimistically.

At the abstract level, the bridge has three layers:

1. a GSLT supplies an operational state space together with its bisimulation quotient;
2. context-decorated HML supplies a language of experiments or observations;
3. quantale weakness measures the mass of distinction events on that quotient.

In slogan form:

> operational behavior gives the ontology, HML gives the observables, and weakness turns distinguishability into evidence.

This is now more than a philosophical analogy in the local Lean state.

Theorem 13.1 (Depth-bounded observational equivalence)**.** *For each modal depth n, the Lean development defines a relation*

$$\mathrm{hmlEquivUpTo}_n(t, u),$$

meaning that no context-decorated HML formula of modal depth at most n distinguishes t from u.

This is formalized in `GSLT/Logic/LogicalMetric.lean`. The same file also defines a binary-valued depth-n logical-distance approximation

$$d_n(t, u) \in \{0, 1\},$$

and proves the corresponding pseudoultrametric inequality.

Theorem 13.2 (Finite-depth witness forces quotient-level distinction under adequacy)**.** *Assume the sound direction of Meredith's adequacy theorem for a GSLT S. If some depth-n context-decorated HML formula distinguishes t from u, then t and u are genuinely distinct in the bisimulation quotient of S. Equivalently, if*

$$d_n(t, u) = 1,$$

then the bisimulation classes $[t]$ and $[u]$ are different.

This is now formalized in `GSLT/Meredith/WeaknessBridge.lean` as `distinguishingWitness_implies_distingui` and `logicalDistanceApprox_eq_one_implies_classes_ne_of_adequacySound`.

Theorem 13.3 (Weakness on the bisimulation quotient)**.** *For a finite bisimulation quotient U equipped with a quantale-valued weight assignment μ, the Lean bridge defines the weakness of the distinction event*

$$\mathrm{weakness}_\mu(\mathrm{distinctionEvent}(U)),$$

interpreting operational distinguishability as an evidence quantity.

This is the content of `GSLT/Meredith/WeaknessBridge.lean`, building on `GSLT/Meredith/Bisimulation.lean`. In the finite probabilistic case, this has exactly the Bennett/Ellerman logical-entropy shape.

The new concrete step is that the abstract minimal-context interface now has an actual process-calculus instance.

Theorem 13.4 (Concrete rho-calculus minimal-context bridge)**.** *The rho-calculus evaluation contexts from* `Languages/ProcessCalculi/RhoCalculus/Context.lean` *now induce a concrete* `HasMinimalContexts` *instance for the rho GSLT, and for each rho context K,*

$$\mathrm{contextStep}(p, K, q) \iff p \rightsquigarrow [K]\, q.$$

This is formalized in `GSLT/Meredith/RhoMinimalContext.lean`. That file also proves concrete HML diamonds for matching COMM interactions.

For the PNP grassroots route, the optimistic use of this material is not that it proves the switching theorem by itself. Rather, it offers a clean semantics for coarse observational views. If a decoder is only sensitive to finite-depth local experiments, then one can hope to represent that by an HML fragment and then ask how much evidence survives under the induced quotient. That is exactly the sort of elegant abstraction that may help disentangle the real compression/locality story from the current proof's more brittle prose. The bridge is now one step tighter than before: finite-depth observational separation no longer just lives as a metric-like approximation, but already certifies distinct quotient classes once adequacy-soundness is available.

The honest limit is equally important: this is still background theory. It does *not* yet produce the switched small class, the generalization theorem, or the K_{poly} contradiction step. But it is now a real machine-checked background layer, not merely an idea. The new obstruction above sharpens this limit: the currently formalized weakness summaries are quotient-level evidence scalars, not a per-instance predictor surface. So any Meredith-style rescue that remains close to the current switching/ERM route still needs an extra layer that turns operational semantics into state-indexed encoded observables.

The next obstruction is sharper still. The new module `InfinitaryHMLObstruction.lean` proves:

1. if $\mathrm{MinimalContext}(S)$ is infinite, then $\mathrm{HMLFormula}(S)$ is infinite;
2. under the same hypothesis, even the depth-1 fragment $\mathrm{DepthFragment}_1(S)$ is infinite;
3. in the concrete rho instance, there is an explicit infinite ladder of certified minimal contexts, so the rho depth-1 HML fragment is already infinite;
4. therefore no finite code space can surject onto the current exact depth-1 probe family, abstractly or in the concrete rho instance.

So the current Meredith semantic layer is now pinned down even more tightly: before one can talk about $\mathrm{polylog}(m)$-bit encoded predictor families, one first needs a separate theorem that compresses or finitizes the relevant observational probe family. The present HML/context layer does not supply that for free.

The Meredith-side trilemma is now simple.

1. Use the current weakness summaries: then the predictor is constant.
2. Use the current exact HML/context probes: then the observable family is already too large, even at depth 1.
3. Use the current exact present-moment surface directly: then one already encounters powerset-like finite probe slices.

So any close-form repair now has to do something genuinely new. It is no longer enough to say "use current weakness" or "use current HML probes a bit more cleverly." One needs a new compressed observational layer together with an explicit theorem showing that the switched predictor factors through it.

The new module `PresentMomentShattering.lean` tightens this further on a very concrete slice of the current rho semantics. Let state i be a single output on channel i, and let environment S be a flat bag of input guards on the channels in a finite list S. Then the file proves:

1. channel i lies in $\mathrm{surfaceChannels}(\mathrm{state}_i, \mathrm{env}_S)$ iff $i \in S$;
2. the concrete external-present-moment pair $(\mathrm{par}(\mathrm{env}_S, \square), i)$ is present iff $i \in S$.

So even before passing to the exact HML layer, the currently formalized state-indexed observational surface already realizes arbitrary finite membership patterns on a natural probe family. This is not yet a full "depth-1 HML shattering" theorem, but it means the large-observable-surface pressure is already visible one layer earlier in the exact present-moment semantics. In fact, the file now packages this as an injective finite-signature map $S \mapsto (i \mapsto \mathrm{observable}_S(i))$ on $\mathrm{Finset}(\mathrm{Fin}\, n)$: distinct finite probe sets yield distinct exact present-moment signatures on the n probe states. Any close-form Meredith rescue now has to compress not only an infinitary exact HML layer, but already a powerset-like concrete probe family in the rho present-moment surface.

14 Kolmogorov Foundations Already Present

The local repository also already contains a substantial Kolmogorov-complexity seed under `Mettapedia/Computabili`

In particular, `Basic.lean` defines:

1. binary strings as `List Bool`,
2. partial algorithms,
3. plain Kolmogorov complexity $C[U](x)$,
4. a universal algorithm built from mathlib's partial-recursive code,
5. the usual invariance theorem for universal algorithms.

This is not yet the time-bounded conditional complexity K_{poly} that the Goertzel program uses. But it matters for the grassroots route because it means the eventual K_{poly} development should be an extension of an existing computability layer rather than a fresh bespoke definition dropped into a late-stage contradiction proof.

In practical terms, the next computability milestone is:

> define conditional and polynomial-time-bounded complexity on top of the existing universal-machine layer, instead of axiomatizing them only at the paper level.

15 What the Grassroots Program Needs Next

The current background theory suggests a fairly crisp optimistic roadmap.

15.1 Finite-Class Learning Layer

The ERM existence layer, finite uniform consistency bound, threshold recovery corollary, rational uniform-rate layer, miss-ratio success bound, and weighted one-predictor and finite-family weighted bounds are now in place. The next finite-class learning steps are:

1. a generalization bound stated in terms of $\log |H|$,
2. a probability interface for i.i.d. or distributional assumptions at the family level,
3. concentration-style control beyond the exact finite-space formulas,
4. a clean interface separating statistical assumptions from coding facts.

In other words, the current Lean state already handles:

1. empirical error on finite samples,
2. existence of ERM predictors in nonempty finite encoded families,
3. exact-fit transfer when some code in the family is sample-consistent,
4. a finite uniform bound on the deceptive sample set,
5. exact ERM recovery of the target off that deceptive set,
6. a strict-cardinality threshold guaranteeing the existence of a good sample,
7. a rational upper bound on the deceptive sample rate,
8. a rational lower bound on the exact recovery rate,
9. strict positivity of that exact recovery rate under the threshold,
10. an explicit miss-ratio success bound of the form $1 - |H|\rho_X^m$,
11. an exact weighted one-predictor law of the form $\text{mass} = \alpha^m$,
12. a weighted family union bound of the form $\text{deceptiveMass} \leq \sum_{h \in H_{\text{bad}}} \alpha_h^m$,
13. a weighted exact recovery bound of the form $\text{recoveryMass} \geq 1 - \sum_{h \in H_{\text{bad}}} \alpha_h^m$.

What remains is to lift that finite uniform rate layer into a real finite-class generalization theorem.

15.2 Concrete Decoder Subclasses

The encoded-family theorem is only an interface. It still needs a nontrivial source of small classes. The likely optimistic candidates are concrete local decoders on tree-like neighborhoods:

1. belief-propagation style update rules,
2. GF(2)-linear constraint summaries from the VV layer,
3. bounded-depth compositions of those primitives.

If those can be encoded uniformly in $\text{polylog}(m)$ bits, then the switching theorem has a plausible landing zone.

15.3 Bridging to the Full PNP Route

Only after those layers are built does it make sense to reconnect to the full proof program:

1. define the distributional ensemble cleanly,
2. define local views as typed accessible surfaces,
3. prove the switched family lies in a small encoded subclass,
4. connect success bounds to a properly formalized K_{poly},
5. finally compare with the upper bound under $P = NP$.

16 Current Status

At present, the grassroots side has several real machine-checked gains and one major pre-existing base layer:

Layer	**Status**	**Artifact**
encoded hypothesis families	proved	`HypothesisClass.lean`
finite-class ERM kernel	proved	`FiniteERM.lean`
finite uniform consistency bound	proved	`FiniteConsistencyBound.lean`
finite uniform recovery threshold	proved	`FiniteUniformRecovery.lean`
finite uniform rate layer	proved	`FiniteUniformRate.lean`
finite uniform success bounds	proved	`FiniteUniformSuccess.lean`
finite weighted one-predictor agreement law	proved	`FiniteIIDAgreement.lean`
finite weighted family union bound	proved	`FiniteIIDFamilyBound.lean`
finite weighted recovery bounds	proved	`FiniteIIDRecovery.lean`
same-route switching/ERM certificate	proved	`SameRouteInterface.lean`
weakness-globality obstruction	proved	`GlobalWeaknessObstruction.lean`
infinitary HML obstruction	proved	`InfinitaryHMLObstruction.lean`
present-moment shattering slice	proved	`PresentMomentShattering.lean`
depth-bounded HML distinction kernel	proved	`LogicalMetric.lean`
GSLT bisimulation-to-weakness bridge	proved	`WeaknessBridge.lean`
concrete rho minimal-context instance	proved	`RhoMinimalContext.lean`
Hyperseed available-region reuse	proved	`HypothesisClass.lean` + `Hyperseed`
plain Kolmogorov infrastructure	already present	`KolmogorovComplexity/Basic.lean`
finite-class generalization bounds	not yet formalized	next target
K_{poly} layer	not yet formalized	later target

This is not the final proof. It is the right sort of groundwork.

17 Conclusion

The grassroots program should stay separate from the crux program.

The crux note asks where the present paper overreaches. This note asks what general machinery we would still want even if Ben patches the overreach later. On that constructive side, the current

Lean result is that small encoded hypothesis families, finite empirical-risk minimizers, finite deceptive-sample bounds, finite uniform recovery thresholds, rational exact-recovery rates, explicit miss-ratio success bounds, weighted one-predictor agreement laws, weighted family deceptive-mass bounds, weighted exact recovery bounds, and observer-relative input surfaces can already be handled cleanly inside one coherent Mettapedia/mathlib formal background layer. Optimistically, the same is now becoming true for the observational-semantics side: finite-depth HML distinction, weakness on operational quotients, and a concrete rho-calculus minimal-context instance are landing as reusable infrastructure rather than as one-off prose. But the current Meredith bridge is now also honest about its own shape: it produces global evidence summaries, not yet a nontrivial per-state learning surface. And even once one moves from those global summaries to the HML/context layer, the current observable family is infinitary by default and already concretely infinitary in the rho instance.

That is enough to sharpen the next optimistic task. We should not try to prove $P \neq NP$ in one jump. We should first prove that the intended switched predictors live in a genuinely small encoded class, and only then lift the rest of the argument on top of that foundation.